

事务处理技术主要包括数据库恢复技术和并发控制技术。本章讨论数据库并发控制的基本概念和实现技术。本章内容有一定的深度和难度。读者在学习本章时一定要做到概念清楚,为此应当认真阅读《概论》中的相应内容,特别是其中的例题,要自己动手先做一做例题,检验是否已经掌握了有关概念。

12.1 基本知识点

数据库是一类共享资源,当多个用户并发地存取数据库时,就会产生多个事务同时存取同一数据的情况。若对并发操作不加控制就可能会存取和存储不正确的数据,破坏事务的一致性和数据库的一致性。所以 DBMS 必须提供并发控制机制。

并发控制机制是衡量一个 DBMS 性能的重要标志之一。

① 需要了解的:数据库并发控制技术的必要性。

② 需要牢固掌握的:掌握并发操作可能产生数据不一致性的情况,包括丢失修改、脏读、不可重复读等,读者要牢固掌握这些情况的确切含义;掌握什么是事务的隔离级别,它与数据不一致性的关系;掌握封锁的类型及不同封锁类型(例如 X 锁、S 锁)的性质和定义,相关的相容控制矩阵;掌握封锁协议的概念;掌握封锁粒度的概念、多粒度封锁方法、多粒度封锁协议的相容控制矩阵。

③ 需要举一反三的:封锁协议与数据一致性的关系;并发调度的可串行性概念;两段锁协议与可串行性的关系、两段锁协议与死锁的关系。

④ 难点:两段锁协议与串行性的关系、与死锁的关系;具有意向锁的多粒度封锁方法的封锁过程。

12.2 习题解答和解析

1. 在数据库中为什么要采用并发控制?并发控制技术能保证事务的哪些特性?

【解析】请认真阅读《概论》第12章12.1节“并发控制概述”相关内容。先读懂[例12.1],

理解并发操作可能带来的数据不一致性问题。

数据库是共享资源，通常有许多个事务同时在运行。当多个事务并发地存取数据库时，就会产生同时读取和/或修改同一数据的情况。若对并发操作不加控制，就可能会存取和存储不正确的数据，破坏数据库的一致性，所以数据库管理系统必须提供并发控制机制。

并发控制可以保证事务的一致性和隔离性。所谓事务的一致性是指，并发事务的执行不会产生数据的不一致性；所谓事务的隔离性是指，一个事务的执行不受其他并发事务的干扰。

2. 并发操作可能会产生哪几类数据不一致？用什么方法能避免各种不一致的情况？

【解析】并发操作带来的数据不一致性主要有丢失修改、脏读、不可重复读、幻读等多种情况。读者要结合《概论》第12章12.1节的例子来理解这些概念。本书主要讲解前面3种不一致性。

① 丢失修改：是指两个事务 T_1 和 T_2 读入同一数据并各自进行修改， T_2 提交的结果破坏（覆盖）了 T_1 提交的结果，导致 T_1 的修改被丢失。

② 脏读：俗称读“脏”数据，是指事务 T_1 修改某一数据并将其写回磁盘，事务 T_2 读取同一数据后， T_1 由于某种原因被撤销，这时被 T_1 修改过的数据恢复原值， T_2 读到的数据就与数据库中的数据不一致，则 T_2 读到的数据就为“脏”数据，即不正确的数据。

③ 不可重复读，是指事务 T_1 读取数据后，事务 T_2 执行更新操作，当事务 T_1 再次读该数据时，得到与前一次不同的值。

④ 幻读，也称作幻影（phantom）现象，是指事务 T_1 读取数据后，事务 T_2 执行插入或删除操作，使 T_1 无法再现前一次读取结果。

幻读包括两种情况：

a. 事务 T_1 按一定条件从数据库中读取某些数据记录后，事务 T_2 删除了其中部分记录，当 T_1 再次按相同条件读取数据时，发现某些记录“神秘地”消失了。

b. 事务 T_1 按一定条件从数据库中读取某些数据记录后，事务 T_2 插入了一些记录，当 T_1 再次按相同条件读取数据时，发现多了一些记录。

避免不一致性的方法就是并发控制。常用的并发控制技术包括封锁技术、时间戳方法、乐观控制方法、多版本并发控制方法等。

3. 事务的隔离级别都有哪些？事务隔离级别与数据一致性的关系是什么？

【解析】请阅读《概论》第12章12.2节“事务的隔离级别”相关内容。事务隔离级别是用来描述 DBMS 对并发操作进行控制的程度。控制越严格，事务的隔离性越强，数据的一致性就越有保障，但系统的效率也会随之下降。用户可以根据实际应用需求，选择事务隔离级别。

SQL 标准给出了事务的4类隔离级别，这4类隔离级别由低到高分别是读未提交、读已提交、可重复读、可串行化。下表（即《概论》第12章表12.1）给出了事务隔离级别与数据不一

致性的关系:

事务隔离级别	数据不一致性			
	丢失修改	脏读	不可重复读	*幻读
读未提交	否	是	是	是
读已提交	否	否	是	是
可重复读	否	否	否	是
可串行化	否	否	否	否

请读者阅读《概论》第 12 章的“扩展阅读：隔离级别实验”二维码内容。该实验演示了在 4 种事务隔离级别下丢失修改、脏读、不可重复读等数据不一致性的情况，读者可以直观感受各种隔离级别在事务中的作用。

4. 什么是封锁？基本的封锁类型有几种？试述它们的含义。

【解析】实现并发控制的方法和技术有很多。封锁是实现并发控制的一种非常重要并且也是最早使用的技术。

封锁是指事务 T 在对某个数据对象（例如表、记录等）操作之前先向系统发出请求，对其加锁。加锁后事务 T 就对该数据对象有了一定的控制，在事务 T 释放其锁之前，其他事务不能更新和/或读取此数据对象。

最基本的封锁类型有两种：排他型锁（简称 X 锁）和共享型锁（简称 S 锁）。

排他型锁又称为写锁。若事务 T 对数据对象 A 加上 X 锁，则只允许 T 读取和修改 A ，其他任何事务都不能再对 A 加任何类型的锁，直到 T 释放 A 上的锁为止。这就保证了其他事务在 T 释放 A 上的锁之前不能再读取和修改 A 。

共享型锁又称为读锁。若事务 T 对数据对象 A 加上 S 锁，则事务 T 可以读 A 但不能修改 A ，其他事务只能再对 A 加 S 锁，而不能加 X 锁，直到 T 释放 A 上的 S 锁为止。这就保证了其他事务可以读 A ，但在 T 释放 A 上的 S 锁之前不能对 A 做任何修改。

5. 如何用封锁机制保证数据的一致性？

【解析】DBMS 按照一定的封锁协议（通俗地讲，协议就是约定）对并发操作进行控制，使得多个并发操作有序地执行，这样就可以避免丢失修改、不可重复读和读“脏”数据等数据不一致性问题。请阅读《概论》第 12 章 12.4 节“封锁协议”相关内容。

下面举例说明 DBMS 在对数据进行读写操作之前，首先对该数据执行封锁操作。下图中事务 T_1 在对 A 进行修改之前先对 A 加 X 锁，记为 Xlock A 。这样，当 T_2 请求对 A 加 X 锁时就被拒绝， T_2 只能等待 T_1 释放 A 上的锁后才能获得对 A 的 X 锁。这时 T_2 读到的 A 是 T_1 更新后的值，再按此新的 A 值进行运算。这样就不会丢失 T_1 的更新。

T_1	T_2
① Xlock A 获得	
② 读 $A=16$	
③ $A \leftarrow A-1$ 写回 $A=15$ Commit Unlock A	Xlock A 等待 等待 等待 等待
④	获得 Xlock A 读 $A=15$ $A \leftarrow A-1$ 写回 $A=14$ Commit Unlock A
⑤	

6. 什么是活锁？试述活锁的产生原因和解决方法。

【解析】如果事务 T_1 封锁了数据 R ，事务 T_2 又请求封锁 R ，于是 T_2 等待； T_3 也请求封锁 R ，当 T_1 释放了 R 上的封锁之后系统首先批准了 T_3 的请求， T_2 仍然等待；然后 T_4 又请求封锁 R ，当 T_3 释放了 R 上的封锁之后系统又批准了 T_4 的请求……， T_2 有可能永远等待，这就是活锁的情形，如下图所示。活锁的含义是某个等待事务等待的时间太长，似乎被锁住了，实际上可以被激活。

T_1	T_2	T_3	T_4
lock R			
	lock R		
	等待	Lock R	
Unlock	等待		Lock R
	等待	Lock R	等待
	等待		等待
	等待	Unlock	等待
	等待		Lock R
	等待		

活锁产生的原因是当一系列封锁不能按照其先后顺序执行时,就可能导致一些事务无限期地等待某个封锁,从而导致活锁。

避免活锁的简单方法是采用先来先服务的策略。当多个事务请求封锁同一数据对象时,封锁子系统按请求封锁的先后次序对事务排队,数据对象上的锁一旦释放,就批准申请队列中第一个事务获得锁。

7. 什么是死锁?请给出预防死锁的若干方法。

【解析】如果事务 T_1 封锁了数据 R_1 , T_2 封锁了数据 R_2 , 然后 T_1 又请求封锁 R_2 , 因 T_2 已封锁了 R_2 , 于是 T_1 等待 T_2 释放 R_2 上的锁。接着 T_2 又申请封锁 R_1 , 因 T_1 已封锁了 R_1 , T_2 也只能等待 T_1 释放 R_1 上的锁。这样就出现了 T_1 在等待 T_2 , 而 T_2 又在等待 T_1 的局面, T_1 和 T_2 两个事务永远不能结束, 形成死锁。

T_1	T_2
lock R_1	
	Lock R_2
Lock R_2	
等待	
等待	Lock R_1
等待	等待

防止死锁的发生, 其实就是要破坏产生死锁的条件。预防死锁通常有两种方法:

① 一次封锁法: 要求每个事务必须一次将所有要使用的数据全部加锁, 否则就不能继续执行。

② 顺序封锁法: 预先对数据对象规定一个封锁顺序, 所有事务都按这个顺序实施封锁。

8. 请给出检测死锁发生的一种方法。当发生死锁后如何解除死锁?

【解析】数据库系统一般不采用预防死锁的方法, 而是允许死锁发生, DBMS 检测到死锁后加以解除。

DBMS 中诊断死锁的方法与计算机操作系统类似, 一般使用超时法或事务等待图法。其中, 超时法是指如果一个事务的等待时间超过了规定的时限, 就认为该事务发生了死锁。

DBMS 并发控制子系统检测到死锁后, 就要设法解除。通常采用的方法是选择一个处理死锁代价最小的事务, 将其撤销, 释放此事务持有的所有锁, 使其他事务得以继续运行下去。

9. 什么样的并发调度是正确的调度?

【解析】可串行化的调度是正确的调度。请阅读《概论》第 12 章 12.6 节“并发调度的可串

行性”相关内容。

可串行化调度的定义：多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行这些事务时的结果相同，称这种调度策略为可串行化调度。

10. 设 T_1 、 T_2 、 T_3 是如下的三个事务 (A 的初值为 0)：

T_1 : $A:=A+2$;

T_2 : $A:=A*2$;

T_3 : $A:=A*A$ (即 $A \leftarrow A^2$)

① 若这三个事务允许并发执行，则有多少种可能的正确结果？请一一列举出来。

【解析】本题是希望读者建立这样的概念：若干个并发事务串行执行可以有多种结果，不同的调度会让并发事务产生不同的结果，只要执行结果等价于串行执行结果中的一个，就认为调度是正确的。这样的调度叫做可串行化调度。

A 的最终结果可能有 2、4、8、16。

因为串行执行次序有 $T_1 T_2 T_3$ 、 $T_1 T_3 T_2$ 、 $T_2 T_1 T_3$ 、 $T_2 T_3 T_1$ 、 $T_3 T_1 T_2$ 、 $T_3 T_2 T_1$ 。

对应的执行结果是 16、8、4、2、4、2。

② 请给出一个可串行化的调度，并给出执行结果。

T_1	T_2	T_3
Slock A		
$Y=A=0$		
Unlock A		
Xlock A		
$A=Y+2$	Slock A	
写回 $A(=2)$	等待	
Unlock A	等待	
	$Y=A=2$	
	Unlock A	
	Xlock A	
	$A=Y*2$	Slock A
	写回 $A(=4)$	等待
	Unlock A	等待
		等待
		$Y=A=4$
		Unlock A
		Xlock A
		$A=Y*2$ 写回 $A(=16)$
		Unlock A

最后结果 A 为 16，是可串行化的调度。

③ 请给出一个非串行化的调度，并给出执行结果。

T_1	T_2	T_3
Slock A		
$Y=A=0$		
Unlock A		
	Slock A	
	$Y=A=0$	
Xlock A		
等待	Unlock A	
$A=Y+2$		
写回 $A(=2)$		Slock A
Unlock A		等待
		$Y=A=2$
		Unlock A
		Xlock A
	Xlock A	
	等待	$Y=Y**2$
	等待	写回 $A(=4)$
	等待	Unlock A
	$A=Y*2$	
	写回 $A(=0)$	
	Unlock A	

最后结果 A 为 0，为非串行化的调度。

【解析】通过这个习题，读者可以发现虽然使用了封锁方法来调度并发事务，但是调度的结果并不正确，是一个非串行化的调度。因此，要探索正确调度的方法，包括如何判定和保证一个调度是可串行化的调度。请读者阅读《概论》第 12 章 12.6 节“并发调度的可串行性”和 12.7 节“两段锁协议”相关内容。

④ 若这三个事务都遵守两段锁协议，请给出一个不产生死锁的可串行化调度。

【解析】两段锁（two-phase lock，简称 2PL）协议是指所有事务必须分两个阶段对数据项加锁和解锁。第一阶段是获得封锁，这个阶段不能释放任何锁；第二阶段是释放封锁，但是不能再申请任何锁。

遵守两段锁协议的调度可以实现并发调度的可串行性，从而保证调度的正确性。

T_1	T_2	T_3
Slock A		
$Y=A=0$		
Xlock A		
$A=Y+2$	Slock A	
写回 $A(=2)$	等待	
Unlock A	等待	
	$Y=A=2$	
	Xlock A	
Unlock A	等待	Slock A
	$A=Y*2$	等待
	写回 $A(=4)$	等待
	Unlock A	等待
		$Y=A=4$
	Unlock A	
		Xlock A
		$A=Y*2$
		写回 $A(=16)$
		Unlock A
		Unlock A

⑤ 若这三个事务都遵守两段锁协议，请给出一个产生死锁的调度。

【解析】遵守两段锁协议的调度是可串行化的调度，遗憾的是遵守两段锁协议的事务可能产生死锁。不过，前面已经讲解了 DBMS 可以用一定的方法检测到死锁，然后加以解除。

遵守两段锁协议的调度可以实现并发调度的可串行性，从而保证调度的正确性。

T_1	T_2	T_3
Slock A		
$Y=A=0$		
	Slock A	
	$Y=A=0$	
Xlock A		
等待		
	Xlock A	
	等待	
		Slock A
		$Y=A=0$
		Xlock A
		等待

11. 今有三个事务的一个调度 $R_3(B) R_1(A) W_3(B) R_2(B) R_2(A) W_2(B) R_1(B) W_1(A)$, 该调度是冲突可串行化的调度吗? 为什么?

【解析】请阅读《概论》第12章12.6.2小节“冲突可串行化调度”相关内容。首先要掌握什么是冲突操作和不冲突操作。不同的事务对同一个数据的读写操作和写写操作称为冲突操作。其他操作是不冲突操作。引入这个概念的目的在于判断一个调度是否为冲突可串行化的调度。

我们找到了这个判断方法: 一个调度 Sc 在保证冲突操作的次序不变的情况下, 通过交换两个事务不冲突操作的次序得到另一个调度 Sc' , 如果 Sc' 是串行的, 称调度 Sc 为冲突可串行化的调度。

本习题给出的调度是冲突可串行化的调度, 理由是:

$$Sc_1 = R_3(B) R_1(A) W_3(B) R_2(B) R_2(A) W_2(B) R_1(B) W_1(A)$$

交换 $R_1(A)$ 和 $W_3(B)$, 得到

$$R_3(B) W_3(B) R_1(A) R_2(B) R_2(A) W_2(B) R_1(B) W_1(A)$$

再交换 $R_1(A)$ 和 $R_2(B) R_2(A) W_2(B)$, 得到

$$Sc_2 = R_3(B) W_3(B) R_2(B) R_2(A) W_2(B) R_1(A) R_1(B) W_1(A)$$

由于 Sc_2 是串行的, 而且两次交换都是基于不冲突操作的, 所以

$$Sc_1 = R_3(B) R_1(A) W_3(B) R_2(B) R_2(A) W_2(B) R_1(B) W_1(A)$$

是冲突可串行化的调度。

读者可以参考阅读《概论》第12章的“微视频: 冲突可串行化判断示例”二维码内容。

12. 试证明若并发事务遵守两段锁协议, 则对这些事务的并发调度是可串行化的。

证明: 首先以两个并发事务 T_1 和 T_2 为例进行证明, 对于存在多个并发事务的情形可以类推。

根据可串行化定义可知, 事务不可串行化只可能发生在下列两种情况:

情况1: 事务 T_1 写某个数据对象 A , T_2 读或写 A 。

情况2: 事务 T_1 读或写某个数据对象 A , T_2 写 A 。

下面称 A 为潜在冲突对象。

设 T_1 和 T_2 访问的潜在冲突的公共对象为 $\{A_1, A_2, \dots, A_n\}$ 。不失一般性, 假设这组潜在冲突对象中 $X = \{A_1, A_2, \dots, A_i\}$ 均符合情况1, $Y = \{A_{i+1}, \dots, A_n\}$ 符合情况2。

$\forall x \in X$, T_1 需要 Xlock x 操作1

T_2 需要 Slock x 或 Xlock x 操作2

① 如果操作1先执行, 则 T_1 获得锁, T_2 等待。

由于遵守两段锁协议, T_1 在成功获得 X 和 Y 中全部对象及非潜在冲突对象的锁后, 才会释放锁。

这时如果 $\exists w \in X$ 或 Y , T_2 已获得 w 的锁, 则出现死锁; 否则, T_1 在对 X 、 Y 中对象全部处理完毕后, T_2 才能执行。这相当于按 T_1 、 T_2 的顺序串行执行。

根据可串行化定义, T_1 和 T_2 的调度是可串行化的。

② 操作 2 先执行的情况与①对称。

因此，若并发事务遵守两段锁协议，在不发生死锁的情况下，对这些事务的并发调度一定是可串行化的。

证毕。

【解析】我们证明了“若并发事务都遵守两段锁协议，则对这些事务的并发调度是可串行化的”。但是，若并发事务的一个调度是可串行化的，不一定所有事务都符合两段锁协议。也就是说，事务遵守两段锁协议是可串行化调度的充分条件，而不是必要条件。读者可以举出示例，就如下面的第 13 题。

13. 举例说明对并发事务的一个调度是可串行化的，而这些并发事务不一定遵守两段锁协议。

【解析】请看下面的示例。事务 T_1 和 T_2 都不遵守两段锁协议。

T_1 : Slock B Unlock B Xlock A Unlock A

T_2 : Slock A Unlock A Xlock B Unlock B

但是对它们并发调度是可串行化的，因为执行结果 $A=3$, $B=4$ 等价于串行执行 T_1 和 T_2 的结果。

T_1	T_2
Slock B	
读 $B=2$	
$Y=B$	
Unlock B	
Xlock A	
	Slock A
	等待
$A=Y+1$	等待
写回 $A=3$	等待
Unlock A	等待
	Slock A
	读 $A=3$
	$X=A$
	Unlock A
	Xlock B
	$B=X+1$
	写回 $B=4$
	Unlock B

14. 考虑如下的调度，说明这些调度集合之间的包含关系。

① 正确的调度。

- ② 可串行化的调度。
- ③ 遵循两段锁(2PL)的调度。
- ④ 串行调度。

【解析】希望读者能够清晰地掌握这些调度的含义，以及它们之间的相互关联。

遵循两段锁(2PL)的调度 $\subset$ 正确的调度 = 可串行化的调度

串行调度 $\subset$ 正确的调度

15. 考虑如下的 T_1 和 T_2 两个事务：

T_1 : R(A); R(B); $B = A + B$; W(B)

T_2 : R(B); R(A); $A = A + B$; W(A)

- ① 改写 T_1 和 T_2 ，增加加锁操作和解锁操作，并要求遵循两段锁协议。
- ② 说明 T_1 和 T_2 的执行是否会引起死锁，给出 T_1 和 T_2 的一个调度并说明之。

【解析】

- ① T_1 和 T_2 都遵循两段锁协议：

T_1	T_2
Slock A	Slock B
R(A)	R(B)
Xlock B	Xlock A
R(B)	R(A)
$B = A + B$	$A = A + B$
W(B)	W(A)
Unlock A	Unlock B
Unlock B	Unlock A

② 如果 T_1 和 T_2 两个并发事务的调度如下，则可能产生死锁： T_1 申请 Xlock B 之后进入等待状态，等待 T_2 释放 B 上的封锁； T_2 申请 Xlock A 之后进入等待状态，等待 T_1 释放 A 上的封锁，于是 T_1 等待 T_2 ， T_2 等待 T_1 ，引起死锁。

T_1	T_2
Slock A	
R(A)	
	Slock B
	R(B)
Xlock B	
等待	Xlock A
	等待

16. 为什么要引进意向锁？意向锁的含义是什么？

【解析】请读者阅读《概论》第12章12.8节“封锁的粒度”相关内容，掌握什么是封锁粒度，什么是多粒度封锁方法。为了提高数据库系统的并发度，提高系统并发控制的效率，DBMS都采用多粒度封锁方法。

引进意向锁的目的是提高封锁子系统的效率。原因是：在多粒度封锁方法中，一个数据对象可能以显式封锁和隐式封锁两种方式加锁，因此系统在对某一数据对象加锁时，不仅要检查该数据对象上是否有（显式和隐式）封锁与之冲突，还要检查其所有上级结点和所有下级结点，看申请的封锁是否与这些结点上的（显式和隐式）封锁冲突。显然，这样的检查方法效率很低。为此引进了意向锁。

意向锁的含义是：对任一结点加锁时，必须先对它的上层结点加意向锁。引进意向锁后，系统对某一数据对象加锁时，不必逐个检查与下一级结点的封锁是否冲突。

17. 试述常用的意向锁（IS 锁、IX 锁、SIX 锁），给出这些锁的相容矩阵。

【解析】请阅读《概论》第12章12.8.2小节“意向锁”相关内容。掌握意向锁的含义：如果对一个结点加意向锁，则说明该结点的后裔结点正在被加锁；对任一结点加锁时，必须先对它的上层结点加意向锁。例如，对任一元组加锁时，必须先对它所在的关系和数据库加意向锁。有了意向锁，数据库管理系统就无须逐个检查下一级结点的显式封锁了。

IS 锁：如果对一个数据对象加 IS 锁，表示它的后裔结点拟（意向）加 S 锁。例如，要对某个元组加 S 锁，则要首先对关系和数据库加 IS 锁。

IX 锁：如果对一个数据对象加 IX 锁，表示它的后裔结点拟（意向）加 X 锁。例如，要对某个元组加 X 锁，则要首先对关系和数据库加 IX 锁。

SIX 锁：如果对一个数据对象加 SIX 锁，表示对它加 S 锁，再加 IX 锁，即 $SIX = S + IX$ 。相容矩阵：如下表所示（《概论》第12章图12.12（a））。

T_1	T_2					
	S	X	IS	IX	SIX	—
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
—	Y	Y	Y	Y	Y	Y

12.3 补充习题及答案

1. 问答题

① 意向锁中为什么存在 SIX 锁，而没有 XIS 锁？

【解析】如果对数据对象加 SIX 锁，表示对它加 S 锁，再加 IX 锁，即对数据对象加 S 锁，后裔结点拟加 X 锁。

X 锁与任何其他类型的锁都不相容，如果数据对象被加上 X 锁，后裔结点则不可能被以任何锁的形式访问，因此 XIS 锁没有意义。

② 完整性约束是否能够保证数据库在处理多个事务时处于一致状态？